

JAVA INTERVIEW PREPARATION

CHAPTER 76

STRUCTURAL PATTERNS AND
FRAMEWORK BOUNDARIES

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Structural Patterns and Framework Boundaries

Senior / Lead Java Developer Reading Chapter

Structural patterns organize relationships between objects, interfaces, and subsystems. They become useful when two useful pieces do not naturally fit, when cross-cutting behavior must surround an operation, or when callers need a smaller view of a complicated subsystem. Their purpose is not to make a class diagram look sophisticated. Their purpose is to preserve a stable boundary while the implementation behind it changes.

For a senior Java developer, the central question is semantic: what promise does the boundary make? An Adapter can translate method names yet still be wrong if it loses retry, timeout, precision, or error semantics. A Proxy can intercept a call yet fail if the call bypasses it. A Decorator can add behavior yet corrupt the result if ordering or duplicate side effects are ignored.

Begin with the boundary pressure

Structural patterns usually answer one of five pressures:

- incompatible contracts must communicate;
- callers need a simpler use-case-oriented surface;
- behavior must be layered without changing the core object;
- access must be controlled or intercepted;
- individual objects and groups must be treated uniformly.

```
business caller
  |
  v
stable application boundary
  |
  |-- translate an external contract      Adapter
  |-- simplify a subsystem               Facade
  |-- add behavior around the same contract Decorator
  |-- control access to a target         Proxy
  |-- represent leaf and group uniformly Composite
```

Several patterns have similar shapes. A wrapper holding another object could be an Adapter, Decorator, or Proxy. Intent and contract distinguish them, not the presence of a field named `delegate`.

Adapter: translate a contract, not merely a method name

An Adapter lets a client use an existing component through the interface the client expects. Object adapters, which hold the adaptee through composition, are normally preferable to inheritance because they can wrap third-party final classes and keep translation explicit.

```

public interface ExchangeRatePort {
    BigDecimal rate(Currency from, Currency to);
}

public final class VendorRateAdapter implements ExchangeRatePort {
    private final VendorFxClient client;

    public VendorRateAdapter(VendorFxClient client) {
        this.client = client;
    }

    @Override
    public BigDecimal rate(Currency from, Currency to) {
        VendorQuote quote = client.fetch(from.getCurrencyCode(),
                                         to.getCurrencyCode());
        if (quote.status() == VendorStatus.NOT_FOUND) {
            throw new RateUnavailableException(from, to);
        }
        return quote.decimalRate().setScale(8, RoundingMode.HALF_EVEN);
    }
}

```

The valuable work is not renaming `fetch` to `rate`. The adapter protects the application from vendor types, maps status codes to domain failures, establishes scale and rounding, and gives tests a controllable port. If the vendor changes its SDK, the domain remains stable.

Translation must preserve meaning. Ask about units, currencies, time zones, nullability, pagination, cancellation, timeouts, idempotency, partial success, and error classification. Converting an HTTP 429 into a generic `IllegalStateException` destroys information needed by a retry policy. Converting milliseconds to seconds with integer truncation may create a valid-looking but incorrect timeout.

At a large integration boundary, a group of adapters can form an anti-corruption layer. It translates the external model into the application's language instead of allowing vendor concepts to spread into entities, services, and database columns.

Adapter placement and ownership

Place the interface on the side that needs stability. In hexagonal architecture, the application defines a port such as `PaymentGateway`; infrastructure supplies an adapter for a particular provider. This inverts the usual dependency: business code does not depend on the vendor SDK.

```

domain/application ---> PaymentGateway port
                        ^
                        |
ProviderAdapter ---> external SDK/API

```

The adapter should also own boundary observability: external operation name, duration, remote result category, and correlation data. It must not log secrets or entire payment payloads. Decide where retry lives. If both the SDK and adapter retry, one application call can multiply into many remote attempts. Make the retry owner and attempt budget explicit.

Test adapters with representative contract fixtures and failure cases, not just mocked success. Provider sandbox tests, schema compatibility tests, and recorded protocol examples catch semantic drift that a unit mock cannot.

Facade: a coherent entrance to a complicated subsystem

A Facade offers a smaller interface for a common use case while coordinating multiple subsystem components. It does not necessarily replace access to every lower-level capability. Its value is that ordinary callers no longer need to know the sequence, collaborators, or internal data representations.

```

public final class OrderPlacementFacade {
    private final Inventory inventory;
    private final Pricing pricing;
    private final Payments payments;
    private final Orders orders;

    public OrderReceipt place(PlaceOrder command) {
        Reservation reservation = inventory.reserve(command.lines());
        Price total = pricing.calculate(command.customer(), command.lines());
        Payment payment = payments.authorize(command.payment(), total);
        return orders.create(command, reservation, payment);
    }
}

```

This example exposes a business operation rather than CRUD access to four internal services. However, it also raises transaction and compensation questions. If payment succeeds and order creation fails, what repairs the partial outcome? A facade simplifies the caller's view; it does not magically make distributed work atomic.

A facade becomes a god service when it accumulates unrelated use cases, owns every rule, and becomes the only route to the subsystem. Keep it cohesive around a capability. Delegate domain decisions to the objects that own them. A facade is different from an adapter: a facade simplifies a subsystem, while an adapter makes one contract look like another. One class can serve both intentions, but naming the responsibilities helps keep them testable.

Decorator: layer behavior while preserving the contract

A Decorator implements the same abstraction as its wrapped component and adds behavior before, after, or around delegation. It supports combinations without creating subclasses such as `TimedCachedAuthorizedCustomerRepository`.

```

public final class TimedCatalog implements Catalog {
    private final Catalog delegate;
    private final MeterRegistry meters;

    @Override
    public Product find(ProductId id) {
        Timer.Sample sample = Timer.start(meters);
        try {
            return delegate.find(id);
        } finally {
            sample.stop(meters.timer("catalog.find"));
        }
    }
}

```

Decorators are useful for metrics, tracing, caching, authorization, validation, auditing, and resilience. The core implementation remains focused on its primary responsibility, and tests can assemble only the layers they need.

Order matters:

```

caller -> authorization -> retry -> metrics -> remote client

authorization outside retry: authorize once
metrics outside retry: measure the whole logical operation
metrics inside retry: measure each physical attempt

```

Neither metric choice is inherently wrong; they answer different questions. A robust system often records both logical-operation latency and attempt latency. Caching outside authorization may leak data if the key does not contain the security context. Retrying outside an audit decorator may create several audit events. Layer order

is part of the behavior and deserves an assembly test.

Decorators should preserve the underlying contract unless the changed semantics are clearly documented. A cache that serves stale results during failure changes freshness. A retry wrapper changes latency and number of side effects. A timeout wrapper changes the possible exceptions. These can be good decisions, but they are not invisible implementation details.

Proxy: control access to a target

A Proxy presents the target's contract while controlling how access occurs. Common forms include remote proxies, lazy-loading proxies, security proxies, virtual proxies for expensive objects, and transactional or tracing proxies.

The structure resembles Decorator, but the intention differs. A decorator primarily adds composable responsibility. A proxy stands in for a target and controls access, identity, location, or lifecycle. In practice, framework-generated objects may do both.

```
caller -> proxy -> target
|
+-- begin transaction
+-- check authorization
+-- initialize lazy target
+-- send remote request
```

Spring commonly implements `@Transactional`, method security, caching, and aspects using proxies. The container gives the caller a proxy rather than the raw bean. The proxy intercepts an eligible method, applies advice, and then invokes the target.

This model explains the self-invocation trap:

```
@Service
class BillingService {
    public void runBatch() {
        chargeOne(); // direct call on this; proxy is bypassed
    }

    @Transactional
    public void chargeOne() { /* database work */ }
}
```

The call from `runBatch` to `chargeOne` does not enter through the external proxy, so the transaction advice on `chargeOne` may not run. Solutions include moving the transactional operation to a separate bean, putting the transaction at the correct public use-case boundary, or using deliberate programmatic transaction management. Injecting a proxy of oneself usually hides a design problem.

JDK dynamic proxies operate through interfaces. Class-based proxies subclass the target and therefore have restrictions around final classes or methods, constructors, and method visibility. Exact details depend on the framework and configuration, so verify them against the version in use. A senior answer connects annotations to the runtime call path rather than treating them as magic.

Remote proxies deserve special caution. A local-looking method can perform network I/O, time out, partially succeed, or return after the caller has abandoned it. Do not design a chatty object interface that hides dozens of remote round trips. Make latency, batching, cancellation, and failure visible in the API.

Composite: treat a part and a whole uniformly

Composite represents individual objects and groups through a common abstraction. It fits trees such as organizational structures, UI components, rule expressions, file-like hierarchies, and product bundles.

```
sealed interface PriceNode permits Item, Bundle {
    Money price();
}

record Item(Money price) implements PriceNode { }

record Bundle(List<PriceNode> children) implements PriceNode {
    Bundle {
        children = List.copyOf(children);
    }

    @Override
    public Money price() {
        return children.stream()
            .map(PriceNode::price)
            .reduce(Money.ZERO, Money::add);
    }
}
```

The caller can price a leaf or an entire tree without type-specific traversal. The record makes child ownership immutable, but construction must still prevent cycles if data can reference existing nodes. Cycles turn recursive traversal into nontermination or stack exhaustion.

The common interface should contain only operations meaningful for both leaves and groups. A method such as `addChild` on every node forces leaves to throw `UnsupportedOperationException`, weakening substitutability. Keep mutation on the composite type or use a builder that validates the tree before publishing an immutable root.

Large composites need explicit performance decisions. Recursive depth can overflow the stack; repeated aggregation may become quadratic; concurrent mutation can produce inconsistent traversal. Iterative traversal, cached aggregates with careful invalidation, or immutable snapshots may be appropriate depending on read/write patterns.

Bridge: separate two dimensions of variation

Bridge separates an abstraction from an implementation so both can vary independently. It is useful when inheritance would create a cross-product of subclasses.

Imagine reports vary by business form and delivery channel. Inheritance tends toward `PdfEmailReport`, `PdfS3Report`, `CsvEmailReport`, and `CsvS3Report`. A bridge composes a `ReportFormat` with a `ReportDestination`.

```
interface ReportDestination {
    void deliver(GeneratedReport report);
}

final class MonthlyReport {
    private final ReportFormat format;
    private final ReportDestination destination;

    void generate(ReportData data) {
        destination.deliver(format.render(data));
    }
}
```

Bridge is most valuable when both axes genuinely change. If there are only two stable implementations, ordinary composition is enough; adding pattern vocabulary does not improve the design.

Structural patterns inside Java and Spring

The Java ecosystem uses these ideas extensively. Streams and I/O wrappers layer behavior in decorator-like chains. JDBC drivers adapt vendor protocols to standard interfaces. Servlet filters and Spring interceptors form ordered chains around requests. Spring repositories often appear behind proxies. HTTP clients may combine adapters, decorators, and remote proxies in one runtime graph.

Recognizing the patterns helps with debugging. If a breakpoint never reaches transaction advice, inspect whether the reference is the proxy and whether the invocation is eligible. If an HTTP call happens three times, inspect every retry layer. If tracing disappears at an adapter, verify context propagation across its thread or protocol boundary.

Do not force framework internals into a single textbook label. Real infrastructure often combines intentions. What matters is being able to explain the call path, ownership, ordering, and failure semantics.

Exception translation across structural boundaries

Every boundary needs an exception policy. Passing every underlying exception through preserves detail but couples callers to implementation. Converting everything into one generic exception protects the interface but destroys the information needed for recovery. The useful middle ground is a small, meaningful failure vocabulary.

An external inventory adapter might distinguish unavailable service, unknown product, optimistic conflict, invalid request, and rate limiting. Preserve the original cause for diagnostics while keeping vendor classes out of the public contract. Mark which failures are retryable only when the boundary has enough evidence to decide. A socket timeout says the response was not received; it does not prove the remote side performed no action.

Decorators and proxies must avoid double wrapping. If each layer creates a new generic exception, operators receive a ten-level cause chain with no stable classification. Translate once at the boundary that changes abstraction level. Add operation context that is safe to log, such as a product identifier or remote operation, while excluding credentials and regulated data.

```
vendor_exception
|
v
adapter classifies at external boundary
|
+-- unavailable / throttled -> resilience policy may retry
+-- rejected input         -> return domain rejection
+-- unknown outcome        -> reconcile by idempotency key
```

Concurrency and thread context

Wrappers are frequently registered as application-scoped objects and used concurrently. A decorator that stores the current request's start time in an instance field creates a race. Request-specific state belongs in local variables or a correctly managed context. Caches need an explicit concurrency strategy, maximum size, expiry policy, and behavior when many requests miss the same key simultaneously.

Asynchronous adapters may move execution to another thread or virtual thread. Logging context, trace context, security identity, locale, and transaction state do not necessarily follow automatically. Define which context is propagated and which must not escape. Thread-local state is particularly easy to misuse when framework interceptors hide the thread transition.

Cancellation should travel through the structural layers. If a caller deadline expires but a proxy leaves the remote request and its retry loop running, the system consumes capacity for work whose result nobody can

use. Java interruption, `CompletableFuture` cancellation, HTTP client cancellation, and reactive cancellation have different guarantees. Test the mechanism actually used by the stack.

Caching as a structural layer

A cache is often implemented as a decorator or proxy, but caching changes the contract in several dimensions. Define the key, freshness, invalidation, negative-result policy, consistency after writes, and behavior when the loader fails.

```
read request
|
cache lookup -- hit --> return value with known freshness
|
miss
v
single-flight loader -> underlying repository/service
|
populate with bounded size and expiry
```

The key must contain every input that affects the result, including tenant, locale, permissions, version, and feature configuration where relevant. Omitting authorization context can become a data leak. Caching null or not-found results may protect a database from repeated misses but can hide a newly created record until expiry.

Avoid a cache stampede by coalescing concurrent loads, adding expiry jitter, or refreshing before expiry. Stale-while-revalidate can improve availability, but the business must accept stale data. Keep caches bounded and emit hit rate, load duration, eviction, size, and failure metrics. A high hit rate is not automatically good if the cached data is wrong or the key cardinality consumes excessive memory.

Latency created by invisible layers

Structural indirection is locally clean but can make an end-to-end call difficult to reason about. A controller might call a security proxy, transaction proxy, cache decorator, retry decorator, metrics decorator, adapter, SDK interceptor, service mesh, and remote proxy. Each layer can add queues, serialization, network calls, or retries.

Create one trace that shows the logical operation and its physical attempts. Use consistent operation names so spans from wrappers compose rather than appear as unrelated calls. Record time spent waiting for a bulkhead or connection separately from remote execution. When a latency incident occurs, operators need to answer whether time was spent in application code, queueing, database checkout, DNS, TLS, remote processing, retry backoff, or response decoding.

A pattern should be removable in a debugger: the team should be able to name every layer and its order. Centralize assembly in a composition root and avoid decorators being added independently by configuration, framework annotations, and SDK defaults without an inventory.

Evolution and compatibility

A stable interface allows implementations to evolve, but interface evolution still needs care. Adding an abstract method breaks implementations; a default method preserves binary compatibility but may impose incorrect semantics. Changing exception types, nullability, blocking behavior, or idempotency can break callers without a compiler error.

Version external adapters independently from domain ports. Contract tests should run against new vendor SDKs before rollout. For a facade used by multiple teams, publish behavior and deprecation policy, not just method signatures. A facade is successful when it lets the subsystem change safely, not when it permanently freezes an accidental first design.

Testing structural compositions

Test a wrapper both in isolation and in the assembled order. Contract tests should run against the raw implementation and each decorated or proxied form where feasible. Verify that successful return values, expected exceptions, cancellation, and resource closure still satisfy the interface.

For adapters, test semantic translation with boundary values: rounding, absent fields, unknown status codes, pagination edges, and time conversions. For facades, test partial failure and compensation. For decorators, test order and exactly-once side effects. For proxies, include a test that obtains the object through the real container so interception is exercised. For composites, test empty trees, deep trees, repeated references, cycles, and aggregate correctness.

Common senior-level traps

- Writing an adapter that translates syntax but loses domain semantics.
- Allowing vendor DTOs and exceptions to leak through the application.
- Retrying in the SDK, adapter, service, and gateway without one attempt budget.
- Turning a facade into a god service that owns unrelated business rules.
- Treating decorator order as incidental.
- Caching outside authorization without a security-aware key.
- Expecting proxy advice on self-invocation or ineligible methods.
- Hiding network calls behind a chatty local-looking proxy.
- Putting child-only operations on every Composite leaf.
- Using Bridge vocabulary where simple composition is already sufficient.
- Adding so many wrapper layers that latency and ownership become invisible.

A strong interview answer

I choose a structural pattern from the boundary pressure. Adapter translates an incompatible external contract into an application-owned port and preserves semantic details such as errors, units, timeouts, and idempotency. Facade exposes a cohesive use case over a complicated subsystem. Decorator composes behavior around the same contract, with ordering treated as part of correctness. Proxy controls access and explains framework behavior such as Spring transactions, including self-invocation limits. Composite models a safe part-whole tree. In production I make the runtime call path observable, define resource ownership, test the assembled wrapper order, and avoid hiding remote latency or partial failure behind a local-looking method.

Chapter summary

Structural patterns protect boundaries and organize collaboration. Their class shapes are less important than their semantic promises. A correct adapter preserves meaning; a cohesive facade simplifies a use case; an ordered decorator chain adds explicit behavior; a proxy controls access through an interceptable call path; a composite makes tree operations uniform without weakening leaf contracts; and a bridge separates independent axes of change. Senior design keeps these layers understandable under failure, concurrency, and production diagnostics.

Revision Notes

- Structural patterns organize relationships and stable boundaries.
- Intent distinguishes Adapter, Decorator, and Proxy even when their shapes match.
- Adapter translates semantics, not only method names.
- Put application-owned ports on the side that needs stability.
- Keep vendor DTOs, status codes, and exceptions behind the adapter.
- Define one retry owner and one end-to-end attempt budget.
- Facade exposes a cohesive use case over a subsystem.
- A facade does not make distributed work atomic.
- Decorator preserves an interface while layering behavior.
- Decorator order changes security, caching, metrics, retry, and audit behavior.
- Proxy controls access, identity, location, or lifecycle of a target.
- Spring advice normally runs only when an eligible call passes through its proxy.
- Self-invocation can bypass transactional, caching, or security advice.
- Remote proxies must not disguise latency and partial failure.
- Composite treats leaves and groups uniformly.
- Keep child mutation off leaf contracts and prevent cycles.
- Bridge separates two independent dimensions of variation.
- Test the real assembled wrapper chain, not only each class alone.

Likely interview question: Adapter versus Facade?

An Adapter changes one contract into the contract a client expects, often protecting the application from an external model. A Facade provides a smaller, cohesive entrance to a complicated subsystem. An integration class may do both, but translation and simplification are distinct responsibilities.

Likely interview question: Decorator versus Proxy?

Both can wrap the same interface. A Decorator primarily composes additional responsibility, often in multiple ordered layers. A Proxy stands in for a target to control access, location, identity, or lifecycle. The runtime object may combine both intentions, so explain behavior rather than relying only on the label.

Likely interview question: Why can Spring self-invocation break transactional behavior?

Proxy-based advice runs when a caller invokes an eligible method through the container-provided proxy. A direct call from one method to another on `this` reaches the target without re-entering that proxy. Put the transaction on the proper external use-case boundary or move the advised operation to another collaborator.

Likely interview question: What makes an Adapter production-ready?

It preserves units, precision, nullability, errors, timeouts, cancellation, idempotency, and pagination semantics; prevents vendor types from leaking; exposes useful low-cardinality telemetry; defines retry ownership; and has contract tests against representative provider behavior.